

Using ANI to Predict Molecular Energies

Introduction

Machine learning has become fundamental in all areas of science, including computational chemistry since it can make reliable approximations about expensive quantum mechanical calculations at much lower computational cost. As explained in the ANI-1 paper, accurate electronic structure methods are often too costly for large molecular systems, whereas faster approximate methods may result in a decline in accuracy.¹ Thus, a good approximation method is often a subtle balance between computational cost and accuracy. Nevertheless, neural networks can learn molecular energies from quantum mechanical reference data that predicts the molecular energies more efficiently after training.¹ This method is often proposed as a solution to the issue between computational cost and accuracy.

In the ANI-1 paper, an extensible neural network method (ANI) that uses atomic environment vectors (AEVs) was introduced by Smith, Isayev, and Roitberg to be trained on DFT energies for organic molecules containing H, C, N, and O atoms. By using AEVs to represent the local chemical environment, the ANI method was able to transform the molecular geometry into symmetry-based features that are invariant to translation, rotation, and permutation of identical atoms.¹ Without AEVs, raw Cartesian coordinates will be fed into the neural network. The transformation process is critical because the model should be transferable, which means that the total energy of a molecule should stay the same while the molecule is moved or rotated.

To predict the total molecular energy, the ANI model uses separate atomic neural networks for each atom type. It produces an atomic energy contribution as each atom's AEV passes through the neural network. It then calculates the total molecular energy by summing the predicted energy contributions over all atoms in the molecule, which makes the model more naturally scale to larger molecules. This method makes the model more transferable and efficient than standard neural networks that use flattened raw Cartesian coordinates or other fixed-size vectors.¹

In this project, TorchANI² was used in a supervised learning ANN model to predict molecular energies from the ANI dataset. The goal was to gain hands-on experience on building and running a training pipeline and to learn the workflow in training a ML model on real-life problems by tuning the model hyperparameters and architecture, testing the stability of the selected model, and finally training the selected model on all data provided in the project. The final result was then compared with the relevant RMSE values reported in the ANI-1 paper to make reasonable evaluations on the performance of the model.

Methods

In the ANI-1 paper, the reported full ANI-1 dataset is about 17.2 million conformations from about 58,000 molecules, with an 80/10/10 split for training, validation, and testing, respectively.¹ However, due to computational limits, the dataset used in this project was only a subset of the full ANI dataset.

The original dataset was obtained from molecules in the GDB database and contains organic molecules with C, H, O, and N atoms.¹ To predict molecular energy, the subset used in this project was first loaded using TorchANI², then self atomic energies were subtracted from the molecular energies. After the chemical species were converted into numerical values in the order H, C, N, and O, the dataset was shuffled.¹ In Checkpoint 2 of the project, the data were split with a proportion of 80/10/10 for training, validation, and testing, respectively, which allows the assessment of model performance on unseen data before tuning the final model.

Before passing through ANI, each atom was transformed into an atomic environment vector using TorchANI's AEV algorithm. The AEV length obtained from the AEV algorithm was 384. ANI used four separate atomic networks for H, C, N, and O, respectively.¹ During training, each atom's AEV were fed into the atomic neural network as the input features, and each atomic network had one hidden layer with hidden dimensions, which is followed by a ReLU activation and a final output layer.¹ The final output from each neural network represents the atomic contribution to the total molecular energy.

Since the project is a regression problem where the model predicts molecular energy values, the atomic neural network training was performed with mean squared error (MSE) loss and Adam optimizer.³ The model was mainly evaluated using mean absolute error (MAE) in kcal/mol. The final training on full data was evaluated using both MAE and root mean square error (RMSE) because relevant values in the ANI-1 paper were reported in RMSE, so RMSE was also computed for a fair comparison.

In Checkpoint 3, the project performed a tuning task to find the best model for the final full data training. The major hyperparameters tested were hidden dimension, learning rate, and L2 regularization, while batch size was fixed to 256. The hidden dimensions were tested with the values 64, 128, and 256. The learning rates were tested with the values 1e-3 and 1e-4. The L2 regularizations were tested with the values 0 and 1e-5. Thus, there were 12 combinations tested. For the comparison between hyperparameters to be fair, the model was reinitialized for each combination before training. This was essential because if the combinations were not reinitialized, later hyperparameter settings would be tested on a model already trained with previous hyperparameter settings.

The best configuration selected after hyperparameter tuning was the one with the lowest validation MAE (≈ 0.987 kcal/mol):

- hidden dimension = 64
- learning rate = 1e-3
- L2 regularization = 0

In Checkpoint 4, this best model was then used to test stability by performing repeated training runs and N-fold cross-validation. Lastly, this model was trained on full data in Checkpoint 5 for 20 epochs, and the final MAE and RMSE were reported as full-data fit metrics, not as held-out test metrics.

Results and Discussion

As shown in Table 1 below, the hyperparameter tuning results from Checkpoint 3, indicated that the model that performed best on the validation set was the one with hidden dimension 64, learning rate 1e-3, and no L2 regularization, which achieved a validation MAE of about 0.987 kcal/mol, the lowest among all other models. Table 1 also showed that validation performance was not improved by increasing the hidden dimension to 128 or 256. The reason might be that larger models usually need more computation and so will not always generalize well on a smaller dataset. In addition, the results showed that the learning rate of 1e-3 performed better than 1e-4. During training, models with 1e-3 also converged faster than the ones using 1e-4. One more information that Table 1 gives was that L2 regularization did not improve performance and generalization for this model and dataset.

Table 1. Hyperparameter tuning results from Checkpoint 3.

	hidden_dim	learning_rate	l2	final_val_loss	train_mae	val_mae
0	64	0.0010	0.00000	0.000005	0.979937	0.987342
1	256	0.0010	0.00000	0.000005	0.983503	0.994686
2	128	0.0010	0.00000	0.000006	1.146211	1.152172
3	256	0.0001	0.00000	0.000011	1.189181	1.201160
4	128	0.0001	0.00000	0.000010	1.326049	1.346883
5	64	0.0001	0.00001	0.000013	1.425070	1.433479
6	256	0.0001	0.00001	0.000013	1.431747	1.442144
7	64	0.0001	0.00000	0.000014	1.515486	1.535375
8	128	0.0010	0.00001	0.000015	1.570841	1.579133
9	128	0.0001	0.00001	0.000017	1.688814	1.693933
10	256	0.0010	0.00001	0.000019	1.720675	1.723294
11	64	0.0010	0.00001	0.000019	1.720143	1.728336

The model selected was then retrained using a 80/10/10 split for train/validation/test and was evaluated on the held-out test set with test MAE around 1.10 kcal/mol. Figure 1 below showed the predicted vs. true energy plot. As shown in Figure 1, most points were close to the diagonal $y = x$ line, which means that the model predictions were close to the reference energies. This indicates that the ANI-style atomic neural network model was able to learn the relationship between molecular structure and reference energy for the course dataset and that the selected model predicts molecular energies with low error.

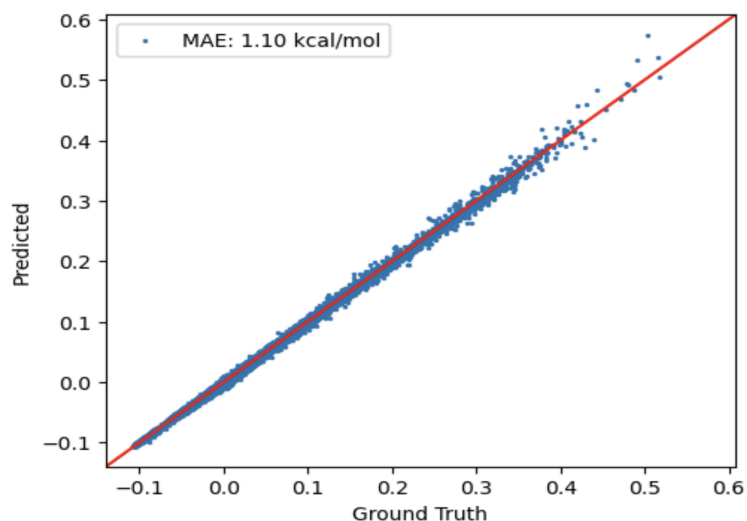


Figure 1. Predicted energy vs. true energy for the selected model from Checkpoint 3. Most points lie close to the $y = x$ line, showing that the selected model predicts molecular energies with low error.

In Checkpoint 4, the stability of best configuration obtained from Checkpoint 3 was tested using multiple runs and 5-fold cross-validation. In the multiple-run experiment, 5 runs were conducted with the same train/validation/test split of 80/10/10 splitting. As shown in Table 2 below, different runs produced slightly different MAE values. This indicates that random initialization and training randomness affect the final model, which is expected since neural networks depend on stochastic optimization. Nevertheless, the average test MAE remained below the 2 kcal/mol test MAE provided in the final project guideline, which indicates that the chosen model is working well.

In the N-fold cross-validation, 5 folds were conducted where the data were split into 5 different train/validation folds. Cross-validation was conducted to confirm that the model was generalized instead of remembering features from any specific train/validation split. As shown in Table 2, the mean training and mean validation errors are almost the same. These results show that the model was reasonably stable and was able to generalize well. Since the purpose of CV is to rotate the validation set across developmental data, the CV algorithm only evaluated training and validation folds, so the statistics for mean test MAE were NaN.

Table 2. Checkpoint 4 multiple-run and cross-validation summary.

	Experiment	mean_Train_MAE	std_Train_MAE	mean_Val_MAE	std_Val_MAE	mean_Test_MAE	std_Test_MAE
0	Multiple Runs	1.754949	0.390384	1.751838	0.390835	1.759738	0.387445
1	5-Fold CV	1.434535	0.353518	1.438651	0.351070	NaN	NaN

In Checkpoint 5, the selected model from Checkpoint 3 was trained on all available data. Since the model was trained on full data, there was no separate held-out test set for this final model. Instead, the final full-data MAE and RMSE were calculated as fit metrics.

As shown in Figure 2 below, the training loss decreased over the 20 final training epochs, indicating that the final model was improving consistently during full-data training.

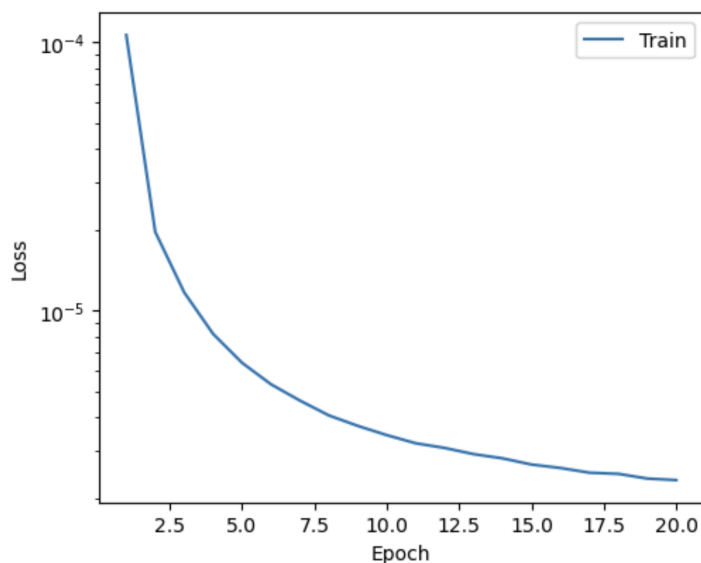


Figure 2. Checkpoint 5 training loss curve. The loss decreases over training.

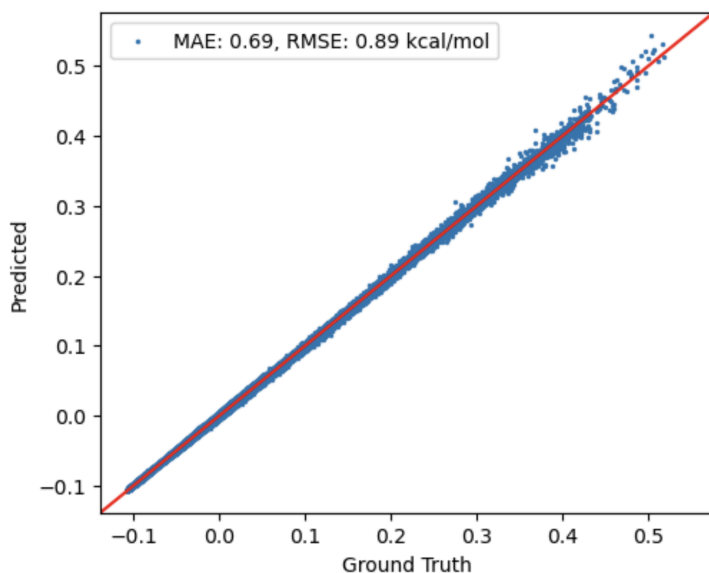


Figure 3. Final Checkpoint 5 full-data prediction plot. The final model was trained on all available data, and the predicted energies were compared with the reference energies. The model achieved a full-data MAE of 0.694 kcal/mol and a full-data RMSE of 0.893 kcal/mol.

As shown in Figure 3 above, the final predicted vs. true energy plot showed that most points lie close to the diagonal $y = x$ line, and the final full-data MAE obtained was 0.694 kcal/mol with a final full-data RMSE of 0.893 kcal/mol, both are relatively low errors. This indicates that the final model fits the project dataset well after training on all available data.

The ANI-1 paper reported the model-fit RMSE values as 1.2 kcal/mol for training, 1.3 kcal/mol for validation, and 1.3 kcal/mol for testing. In comparison to these values, the final full-data RMSE of 0.893 kcal/mol is lower. However, this is not a one-to-one comparison because the full-data RMSE was evaluated on one full dataset while the model-fit RMSE reported by ANI-1 paper was split-based values on a larger and more diverse dataset of about 17.2 million conformations from about 58,000 molecules. The ANI-1 model was also more complex and had a much larger data-generation pipeline. Thus, the lower full-data RMSE does not represent a better model in comparison to the ANI-1 model. It only indicates that the selected model works well on the course dataset, and can be seen as a successful implementation of the ANI idea.

Overall, the validation, test and CV results from Checkpoint 4, combined with the full-data training result in Checkpoint 5, supported that the selected model learned the dataset patterns sufficient enough that the model was able to predict molecular energies with relatively low error. This supports the effectiveness of AEV-based atomic neural networks for molecular energy prediction.

Conclusion

In conclusion, this project successfully reproduced the main ANI idea that molecular energies can be predicted by converting local atomic environments into AEVs, passing them through atom-specific neural networks, and summing atomic energy contributions. In Checkpoint 3, hyperparameter tuning selected the best configuration of hidden dimension 64, learning rate $1e-3$, and no L2 regularization with the lowest validation MAE among all other configurations. The selected model was then retrained using a 80/10/10 split and evaluated on the held-out test set with test MAE around 1.10 kcal/mol. The selected model also showed reasonable stability in multiple-run and cross-validation experiments from Checkpoint 4. Lastly, the selected model was trained on all available data and achieved a full-data MAE of 0.694 kcal/mol and a full-data RMSE of 0.893 kcal/mol. Even though this result is not appropriate for direct comparison to the model-fit RMSE values reported in the ANI-1 paper, it still Overall, the results show that the ANI framework can successfully learn molecular energy patterns from AEV features and produce predictions on the course dataset with high accuracy.

References

1. Smith, J. S., Isayev, O., & Roitberg, A. E. (2017). ANI-1: An extensible neural network potential with DFT accuracy at force field computational cost. *Chemical Science*, 8, 3192–3203.
2. Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., & Roitberg, A. E. (2020). TorchANI: A free and open source PyTorch based deep learning implementation of the ANI neural network potentials. *Journal of Chemical Information and Modeling*, 60(7), 3408–3415.
3. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980.

Final Project

This notebook is adapted from here:

https://aiqm.github.io/torchani/examples/nnp_training.html

Checkpoint 1: Data preparation

1. Create a working directory:

`/global/scratch/users/[USER_NAME]/[DIR_NAME]` . Replace the `[USER_NAME]` with yours and specify a `[DIR_NAME]` you like.

2. Copy the Jupyter Notebook to the working directory
3. Download the ANI dataset `ani_dataset_gdb_s01_to_s04.h5` from bCourses and upload it to the working directory

```
In [2]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torchani
```

Use GPU

```
In [3]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
```

cuda

Set up AEV computer

AEV: Atomic Environment Vector (atomic features)

Ref: Chem. Sci., 2017, 8, 3192

```
In [4]: def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5
    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)
```

```

EtaA = torch.tensor([8], dtype=torch.float, device=device)
Zeta = torch.tensor([32], dtype=torch.float, device=device)
ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float,
device=device)
ShfZ = torch.tensor([
    0.19634954, 0.58904862, 0.9817477, 1.37444680,
    1.76714590, 2.15984490, 2.5525440, 2.94524300
], dtype=torch.float, device=device)

num_species = 4
aev_computer = torchani.AEVComputer(
    Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, num_species
)
return aev_computer

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print(aev_dim)

```

384

Prepare dataset & split

```

In [5]: def load_anl_dataset(dspath):
        self_energies = torch.tensor([
            0.500607632585, -37.8302333826,
            -54.5680045287, -75.0362229210
        ], dtype=torch.float, device=device)
        energy_shifter = torchani.utils.EnergyShifter(None)
        species_order = ['H', 'C', 'N', 'O']

        dataset = torchani.data.load(dspath)
        dataset = dataset.subtract_self_energies(energy_shifter, species_order)
        dataset = dataset.species_to_indices(species_order)
        dataset = dataset.shuffle()
        return dataset

dataset = load_anl_dataset("./ani_gdb_s01_to_s04.h5")
# Use dataset.split method to do split
train_data, val_data, test_data = dataset.split(0.8, 0.1, None)

```

Batching

```

In [6]: batch_size = 8192
        # use dataset.collate(...).cache() method to do batching
        train_data_loader = train_data.collate(batch_size).cache()
        val_data_loader = val_data.collate(batch_size).cache()
        test_data_loader = test_data.collate(batch_size).cache()

```

Torchani API

```
In [7]: class AtomicNet(nn.Module):
    def __init__(self, hidden_dim=128):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, x):
        return self.layers(x)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
```

```
In [8]: train_data_batch = next(iter(train_data_loader))

loss_func = nn.MSELoss()
species = train_data_batch['species'].to(device)
coords = train_data_batch['coordinates'].to(device)
true_energies = train_data_batch['energies'].to(device).float()
_, pred_energies = model((species, coords))
loss = loss_func(true_energies, pred_energies)
print(loss)
```

tensor(0.1107, device='cuda:0', grad_fn=<MseLossBackward0>)

Checkpoint 2 : Network construction and workflow development

```
In [9]: import matplotlib.pyplot as plt
import copy
```

```
In [20]: class ANITrainer:
    def __init__(self, model, batch_size, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.l2 = l2
        self.batch_size = batch_size
```

```

        self.optimizer = torch.optim.Adam(model.parameters(),
learning_rate, weight_decay=self.l2)
        self.epoch = epoch

    def train(self, train_data, val_data, early_stop=True, draw_curve=True,
max_train_batches=None, max_val_batches=None):
        self.model.train()

        # init data loader
        print("Initialize training data...")
        train_data_loader = train_data.collate(self.batch_size).cache()

        # definition of loss function: MSE is a good choice!
        loss_func = nn.MSELoss()

        # record epoch losses
        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf
        weights = copy.deepcopy(self.model.state_dict())

        for i in tqdm(range(self.epoch), leave=True):
            self.model.train()
            train_epoch_loss = 0.0
            total_samples = 0

            for batch_idx, train_data_batch in
enumerate(train_data_loader):
                if max_train_batches is not None and batch_idx >=
max_train_batches:
                    break

                # compute energies
                species = train_data_batch['species'].to(device)
                coords = train_data_batch['coordinates'].to(device)
                true_energies =
train_data_batch['energies'].to(device).float()
                _, pred_energies = self.model((species, coords))

                # compute loss
                batch_loss = loss_func(pred_energies, true_energies)

                # do a step
                self.optimizer.zero_grad()
                batch_loss.backward()
                self.optimizer.step()

                batch_importance = species.shape[0]
                train_epoch_loss += batch_loss.item() * batch_importance
                total_samples += batch_importance

            train_epoch_loss /= total_samples

            # # use the self.evaluate to get loss on the validation set
            # val_epoch_loss = self.evaluate(val_data, draw_plot=False,
max_batches=max_val_batches)

```

```

        # # append the losses
        # train_loss_list.append(train_epoch_loss)
        # val_loss_list.append(val_epoch_loss)

        # print(f"Epoch {i+1}/{self.epoch} | Train Loss:
{train_epoch_loss:.6f} | Val Loss: {val_epoch_loss:.6f}")

        # if early_stop:
        #     if val_epoch_loss < lowest_val_loss:
        #         lowest_val_loss = val_epoch_loss
        #         weights = copy.deepcopy(self.model.state_dict())

        # Checkpoint 5 updated code for training all data:
        if val_data is not None:
            val_epoch_loss = self.evaluate(val_data, draw_plot=False,
max_batches=max_val_batches)
            val_loss_list.append(val_epoch_loss)
            print(f"Epoch {i+1}/{self.epoch} | Train Loss:
{train_epoch_loss:.6f} | Val Loss: {val_epoch_loss:.6f}")

            if early_stop and val_epoch_loss < lowest_val_loss:
                lowest_val_loss = val_epoch_loss
                weights = copy.deepcopy(self.model.state_dict())
            else:
                print(f"Epoch {i+1}/{self.epoch} | Train Loss:
{train_epoch_loss:.6f}")

            train_loss_list.append(train_epoch_loss)

        if draw_curve:
            fig, ax = plt.subplots(1, 1, figsize=(5, 4),
constrained_layout=True)
            ax.set_yscale("log")
            # Plot train loss and validation loss
            ax.plot(range(1, len(train_loss_list) + 1), train_loss_list,
label='Train')
            # ax.plot(range(1, len(val_loss_list) + 1), val_loss_list,
label='Validation')
            if len(val_loss_list) > 0:
                ax.plot(range(1, len(val_loss_list) + 1), val_loss_list,
label='Validation')
            ax.legend()
            ax.set_xlabel("Epoch")
            ax.set_ylabel("Loss")

        if early_stop:
            self.model.load_state_dict(weights)

        return train_loss_list, val_loss_list

    def evaluate(self, data, draw_plot=False, max_batches=None,
return_mae=False, return_rmse=False):

        # init data loader

```

```

data_loader = data.collate(self.batch_size).cache()

# init loss function
loss_func = nn.MSELoss()
total_loss = 0.0
total_samples = 0

if draw_plot or return_mae or return_rmse:
    true_energies_all = []
    pred_energies_all = []

self.model.eval()
with torch.no_grad():
    for batch_idx, batch_data in enumerate(data_loader):
        if max_batches is not None and batch_idx >= max_batches:
            break

        # compute energies
        species = batch_data['species'].to(device)
        coords = batch_data['coordinates'].to(device)
        true_energies = batch_data['energies'].to(device).float()
        _, pred_energies = self.model((species, coords))

        # compute loss
        batch_loss = loss_func(pred_energies, true_energies)

        batch_importance = species.shape[0]
        total_loss += batch_loss.item() * batch_importance
        total_samples += batch_importance

        if draw_plot or return_mae or return_rmse:
            true_energies_all.append(true_energies.detach().cpu().numpy().flatten())
            pred_energies_all.append(pred_energies.detach().cpu().numpy().flatten())

total_loss /= total_samples

if draw_plot or return_mae or return_rmse:
    true_energies_all = np.concatenate(true_energies_all)
    pred_energies_all = np.concatenate(pred_energies_all)
    # Report the mean absolute error
    # The unit of energies in the dataset is hartree
    # please convert it to kcal/mol when reporting the mean
    absolute error
    # 1 hartree = 627.5094738898777 kcal/mol
    # MAE = mean(|true - pred|)
    hartree2kcalmol = 627.5094738898777

    mae = np.mean(np.abs(true_energies_all - pred_energies_all)) *
    hartree2kcalmol
    rmse = np.sqrt(np.mean((true_energies_all - pred_energies_all)
    ** 2)) * hartree2kcalmol

    if draw_plot:
        fig, ax = plt.subplots(1, 1, figsize=(5, 4),

```

```

constrained_layout=True)
    ax.scatter(true_energies_all, pred_energies_all,
label=f"MAE: {mae:.2f}, RMSE: {rmse:.2f} kcal/mol", s=2)
    ax.set_xlabel("Ground Truth")
    ax.set_ylabel("Predicted")
    xmin, xmax = ax.get_xlim()
    ymin, ymax = ax.get_ylim()
    vmin, vmax = min(xmin, ymin), max(xmax, ymax)
    ax.set_xlim(vmin, vmax)
    ax.set_ylim(vmin, vmax)
    ax.plot([vmin, vmax], [vmin, vmax], color='red')
    ax.legend()

    if return_mae and return_rmse:
        return mae, rmse

    if return_mae:
        return mae

    if return_rmse:
        return rmse

return total_loss

```

Train a small subset of the data:

```

In [12]: net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torchANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)

```

```

In [13]: trainer = ANITrainer(
    model,
    batch_size=128,
    learning_rate=1e-3,
    epoch=3,
    l2=1e-5
)

train_loss_list, val_loss_list = trainer.train(
    train_data,
    val_data,
    early_stop=True,
    draw_curve=True,
    max_train_batches=5,

```

```
max_val_batches=2
)
```

Sequential - Number of parameters: 197636

Initialize training data...

33%|██████| 1/3 [00:24<00:48, 24.32s/it]

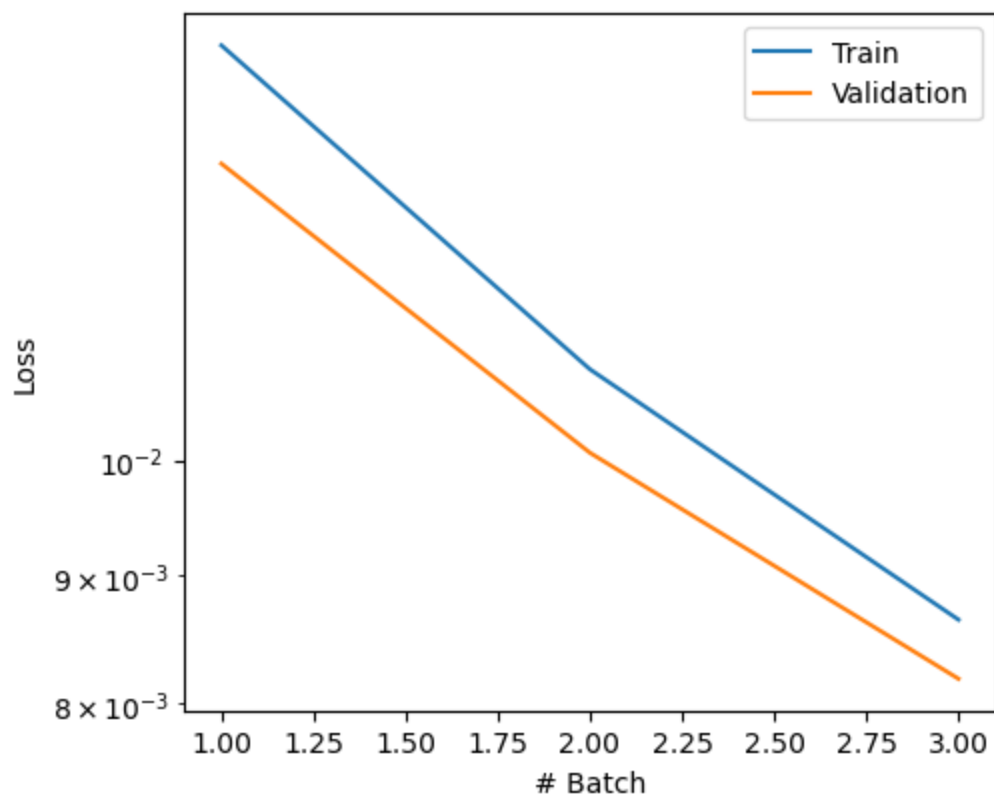
Epoch 1/3 | Train Loss: 0.014665 | Val Loss: 0.013151

67%|██████| 2/3 [00:26<00:11, 11.44s/it]

Epoch 2/3 | Train Loss: 0.010882 | Val Loss: 0.010076

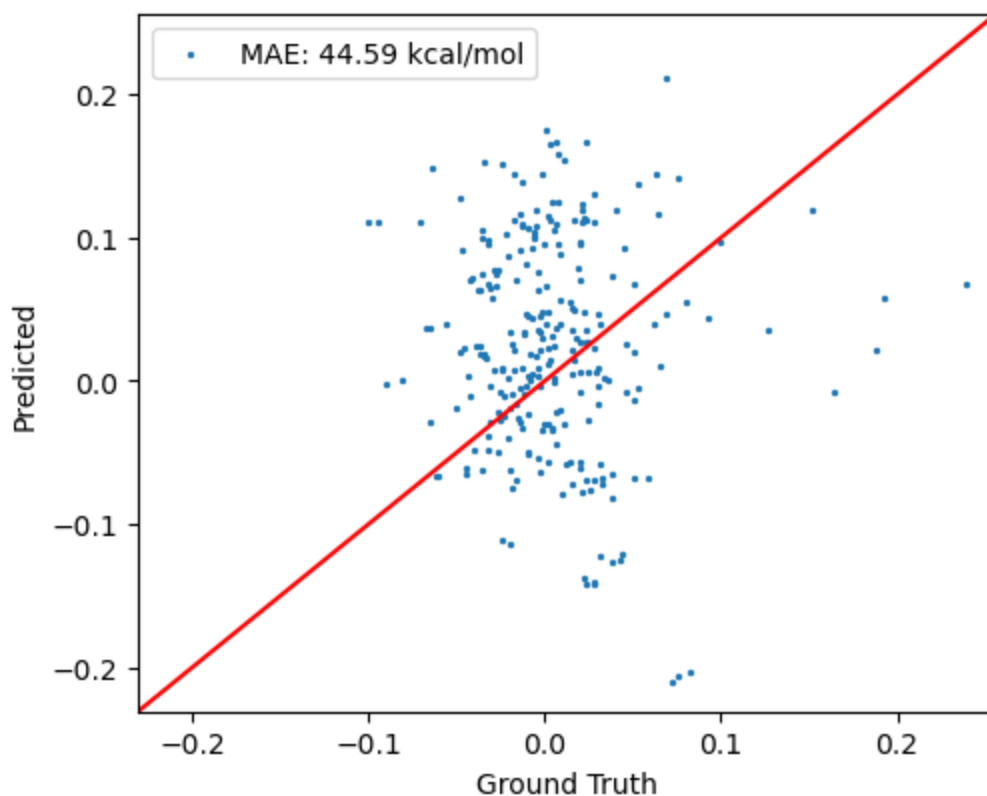
100%|██████| 3/3 [00:30<00:00, 10.03s/it]

Epoch 3/3 | Train Loss: 0.008640 | Val Loss: 0.008183



```
In [14]: trainer.evaluate(val_data, draw_plot=True, max_batches=2)
```

```
Out[14]: 0.008182576857507229
```



The ANI model was trained using MSE loss (for solving regression problems) and Adam optimizer (because it works more efficiently, the tutorial also used Adam as the default optimizer).

Both training and validation losses decrease over epochs, indicating that the model is learning.

The evaluation shows a relatively high MAE (Mean Absolute Error more suitable for evaluation), which is expected due to training on a small subset with small amount of epochs.

So, overall, the training and evaluation pipeline works correctly.

Checkpoint 3: Regularization strategies and hyperparameter tuning

```
In [10]: # hidden_dims = [128, 256]
# learning_rates = [1e-3, 1e-4]
# l2_values = [0, 1e-5]
# epochs = 10

hidden_dims = [64, 128, 256]
learning_rates = [1e-3, 1e-4]
l2_values = [0, 1e-5]
epochs = 5
```

```

In [11]: results = []

for hidden_dim in hidden_dims:
    for lr in learning_rates:
        for l2 in l2_values:

            print(f"\nRunning: hidden={hidden_dim}, lr={lr}, l2={l2}")

            # reinitialize model
            net_H = AtomicNet(hidden_dim)
            net_C = AtomicNet(hidden_dim)
            net_N = AtomicNet(hidden_dim)
            net_O = AtomicNet(hidden_dim)

            ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
            model = nn.Sequential(aev_computer, ani_net).to(device)

            trainer = ANITrainer(
                model,
                batch_size=256,
                learning_rate=lr,
                epoch=epochs,
                l2=l2
            )

            train_loss, val_loss = trainer.train(
                train_data,
                val_data,
                draw_curve=False
            )

            final_val_loss = val_loss[-1]

            train_mae = trainer.evaluate(train_data, return_mae=True)
            val_mae = trainer.evaluate(val_data, return_mae=True)

            print(f"Train MAE: {train_mae:.2f} kcal/mol | Val MAE:
{val_mae:.2f} kcal/mol")

            results.append({
                "hidden_dim": hidden_dim,
                "learning_rate": lr,
                "l2": l2,
                "final_val_loss": final_val_loss,
                "train_mae": train_mae,
                "val_mae": val_mae
            })

```

Running: hidden=64, lr=0.001, l2=0

Sequential – Number of parameters: 98820

Initialize training data...

20%|██████| 1/5 [01:41<06:45, 101.47s/it]

Epoch 1/5 | Train Loss: 0.000140 | Val Loss: 0.000032

40%|██████| 2/5 [03:23<05:05, 101.71s/it]

Epoch 2/5 | Train Loss: 0.000024 | Val Loss: 0.000020

```

60%|██████    | 3/5 [05:03<03:22, 101.19s/it]
Epoch 3/5 | Train Loss: 0.000015 | Val Loss: 0.000010
80%|██████    | 4/5 [06:44<01:40, 100.80s/it]
Epoch 4/5 | Train Loss: 0.000010 | Val Loss: 0.000008
100%|██████████| 5/5 [08:24<00:00, 100.85s/it]
Epoch 5/5 | Train Loss: 0.000008 | Val Loss: 0.000005
Train MAE: 0.98 kcal/mol | Val MAE: 0.99 kcal/mol

```

Running: hidden=64, lr=0.001, l2=1e-05
 Sequential – Number of parameters: 98820
 Initialize training data...

```

20%|██        | 1/5 [02:00<08:03, 120.91s/it]
Epoch 1/5 | Train Loss: 0.000324 | Val Loss: 0.000030
40%|████      | 2/5 [04:48<07:24, 148.12s/it]
Epoch 2/5 | Train Loss: 0.000046 | Val Loss: 0.000020
60%|██████    | 3/5 [08:02<05:38, 169.22s/it]
Epoch 3/5 | Train Loss: 0.000033 | Val Loss: 0.000022
80%|██████    | 4/5 [11:49<03:12, 192.00s/it]
Epoch 4/5 | Train Loss: 0.000027 | Val Loss: 0.000016
100%|██████████| 5/5 [16:02<00:00, 192.60s/it]
Epoch 5/5 | Train Loss: 0.000024 | Val Loss: 0.000019
Train MAE: 1.72 kcal/mol | Val MAE: 1.73 kcal/mol

```

Running: hidden=64, lr=0.0001, l2=0
 Sequential – Number of parameters: 98820
 Initialize training data...

```

20%|██        | 1/5 [01:46<07:05, 106.31s/it]
Epoch 1/5 | Train Loss: 0.003423 | Val Loss: 0.000141
40%|████      | 2/5 [03:33<05:19, 106.61s/it]
Epoch 2/5 | Train Loss: 0.000084 | Val Loss: 0.000053
60%|██████    | 3/5 [05:19<03:33, 106.72s/it]
Epoch 3/5 | Train Loss: 0.000037 | Val Loss: 0.000026
80%|██████    | 4/5 [07:07<01:46, 106.92s/it]
Epoch 4/5 | Train Loss: 0.000023 | Val Loss: 0.000018
100%|██████████| 5/5 [08:53<00:00, 106.79s/it]
Epoch 5/5 | Train Loss: 0.000017 | Val Loss: 0.000014
Train MAE: 1.52 kcal/mol | Val MAE: 1.54 kcal/mol

```

Running: hidden=64, lr=0.0001, l2=1e-05
 Sequential – Number of parameters: 98820
 Initialize training data...

```

20%|██        | 1/5 [01:44<06:58, 104.55s/it]
Epoch 1/5 | Train Loss: 0.000369 | Val Loss: 0.000048
40%|████      | 2/5 [03:31<05:18, 106.04s/it]
Epoch 2/5 | Train Loss: 0.000034 | Val Loss: 0.000025
60%|██████    | 3/5 [05:18<03:32, 106.41s/it]
Epoch 3/5 | Train Loss: 0.000022 | Val Loss: 0.000019
80%|██████    | 4/5 [07:05<01:46, 106.77s/it]
Epoch 4/5 | Train Loss: 0.000018 | Val Loss: 0.000015
100%|██████████| 5/5 [08:54<00:00, 106.86s/it]

```

Epoch 5/5 | Train Loss: 0.000015 | Val Loss: 0.000013

Train MAE: 1.43 kcal/mol | Val MAE: 1.43 kcal/mol

Running: hidden=128, lr=0.001, l2=0

Sequential – Number of parameters: 197636

Initialize training data...

20%|██████ | 1/5 [01:58<07:55, 118.87s/it]

Epoch 1/5 | Train Loss: 0.000422 | Val Loss: 0.000017

40%|██████ | 2/5 [03:57<05:57, 119.00s/it]

Epoch 2/5 | Train Loss: 0.000041 | Val Loss: 0.000011

60%|██████ | 3/5 [05:52<03:53, 116.96s/it]

Epoch 3/5 | Train Loss: 0.000023 | Val Loss: 0.000010

80%|██████ | 4/5 [07:45<01:55, 115.27s/it]

Epoch 4/5 | Train Loss: 0.000015 | Val Loss: 0.000007

100%|██████ | 5/5 [09:36<00:00, 115.34s/it]

Epoch 5/5 | Train Loss: 0.000011 | Val Loss: 0.000006

Train MAE: 1.15 kcal/mol | Val MAE: 1.15 kcal/mol

Running: hidden=128, lr=0.001, l2=1e-05

Sequential – Number of parameters: 197636

Initialize training data...

20%|██████ | 1/5 [02:39<10:39, 159.90s/it]

Epoch 1/5 | Train Loss: 0.000167 | Val Loss: 0.000024

40%|██████ | 2/5 [06:56<10:49, 216.57s/it]

Epoch 2/5 | Train Loss: 0.000037 | Val Loss: 0.000018

60%|██████ | 3/5 [12:26<08:56, 268.47s/it]

Epoch 3/5 | Train Loss: 0.000028 | Val Loss: 0.000017

80%|██████ | 4/5 [19:03<05:19, 319.10s/it]

Epoch 4/5 | Train Loss: 0.000023 | Val Loss: 0.000017

100%|██████ | 5/5 [26:34<00:00, 318.92s/it]

Epoch 5/5 | Train Loss: 0.000021 | Val Loss: 0.000015

Train MAE: 1.57 kcal/mol | Val MAE: 1.58 kcal/mol

Running: hidden=128, lr=0.0001, l2=0

Sequential – Number of parameters: 197636

Initialize training data...

20%|██████ | 1/5 [02:02<08:10, 122.64s/it]

Epoch 1/5 | Train Loss: 0.001766 | Val Loss: 0.000077

40%|██████ | 2/5 [04:06<06:09, 123.27s/it]

Epoch 2/5 | Train Loss: 0.000046 | Val Loss: 0.000030

60%|██████ | 3/5 [06:10<04:07, 123.83s/it]

Epoch 3/5 | Train Loss: 0.000024 | Val Loss: 0.000018

80%|██████ | 4/5 [08:14<02:03, 123.88s/it]

Epoch 4/5 | Train Loss: 0.000017 | Val Loss: 0.000016

100%|██████ | 5/5 [10:18<00:00, 123.66s/it]

Epoch 5/5 | Train Loss: 0.000014 | Val Loss: 0.000010

Train MAE: 1.33 kcal/mol | Val MAE: 1.35 kcal/mol

Running: hidden=128, lr=0.0001, l2=1e-05

Sequential – Number of parameters: 197636

Initialize training data...

20%	██████	1/5 [02:05<08:23, 125.91s/it]
Epoch 1/5		Train Loss: 0.000169 Val Loss: 0.000033
40%	██████	2/5 [04:10<06:15, 125.27s/it]
Epoch 2/5		Train Loss: 0.000026 Val Loss: 0.000026
60%	██████	3/5 [06:16<04:10, 125.33s/it]
Epoch 3/5		Train Loss: 0.000019 Val Loss: 0.000018
80%	██████	4/5 [08:23<02:06, 126.24s/it]
Epoch 4/5		Train Loss: 0.000016 Val Loss: 0.000016
100%	██████	5/5 [10:35<00:00, 127.19s/it]
Epoch 5/5		Train Loss: 0.000014 Val Loss: 0.000017
Train MAE: 1.69 kcal/mol Val MAE: 1.69 kcal/mol		

Running: hidden=256, lr=0.001, l2=0

Sequential – Number of parameters: 395268

Initialize training data...

20%	██████	1/5 [02:37<10:29, 157.31s/it]
Epoch 1/5		Train Loss: 0.000425 Val Loss: 0.000038
40%	██████	2/5 [05:03<07:31, 150.61s/it]
Epoch 2/5		Train Loss: 0.000050 Val Loss: 0.000011
60%	██████	3/5 [07:24<04:52, 146.29s/it]
Epoch 3/5		Train Loss: 0.000025 Val Loss: 0.000009
80%	██████	4/5 [09:42<02:23, 143.01s/it]
Epoch 4/5		Train Loss: 0.000015 Val Loss: 0.000010
100%	██████	5/5 [11:58<00:00, 143.60s/it]
Epoch 5/5		Train Loss: 0.000010 Val Loss: 0.000005
Train MAE: 0.98 kcal/mol Val MAE: 0.99 kcal/mol		

Running: hidden=256, lr=0.001, l2=1e-05

Sequential – Number of parameters: 395268

Initialize training data...

20%	██████	1/5 [03:25<13:41, 205.38s/it]
Epoch 1/5		Train Loss: 0.000400 Val Loss: 0.000178
40%	██████	2/5 [10:05<16:00, 320.11s/it]
Epoch 2/5		Train Loss: 0.000056 Val Loss: 0.000020
60%	██████	3/5 [18:47<13:44, 412.26s/it]
Epoch 3/5		Train Loss: 0.000034 Val Loss: 0.000016
80%	██████	4/5 [29:31<08:23, 503.48s/it]
Epoch 4/5		Train Loss: 0.000027 Val Loss: 0.000018
100%	██████	5/5 [42:47<00:00, 608.96s/it]
Epoch 5/5		Train Loss: 0.000022 Val Loss: 0.000019
100%	██████	5/5 [42:47<00:00, 513.50s/it]
Train MAE: 1.72 kcal/mol Val MAE: 1.72 kcal/mol		

Running: hidden=256, lr=0.0001, l2=0

Sequential – Number of parameters: 395268

Initialize training data...

```

20%|██████    | 1/5 [02:46<11:06, 166.74s/it]
Epoch 1/5 | Train Loss: 0.000174 | Val Loss: 0.000026
40%|██████    | 2/5 [05:30<08:15, 165.25s/it]
Epoch 2/5 | Train Loss: 0.000021 | Val Loss: 0.000016
60%|██████    | 3/5 [08:16<05:31, 165.61s/it]
Epoch 3/5 | Train Loss: 0.000015 | Val Loss: 0.000010
80%|██████    | 4/5 [11:02<02:45, 165.46s/it]
Epoch 4/5 | Train Loss: 0.000012 | Val Loss: 0.000008
100%|███████  | 5/5 [13:46<00:00, 165.24s/it]
Epoch 5/5 | Train Loss: 0.000010 | Val Loss: 0.000011
Train MAE: 1.19 kcal/mol | Val MAE: 1.20 kcal/mol

```

Running: hidden=256, lr=0.0001, l2=1e-05
 Sequential – Number of parameters: 395268
 Initialize training data...

```

20%|██████    | 1/5 [02:47<11:11, 167.98s/it]
Epoch 1/5 | Train Loss: 0.000191 | Val Loss: 0.000031
40%|██████    | 2/5 [05:39<08:29, 169.91s/it]
Epoch 2/5 | Train Loss: 0.000028 | Val Loss: 0.000022
60%|██████    | 3/5 [08:34<05:44, 172.24s/it]
Epoch 3/5 | Train Loss: 0.000021 | Val Loss: 0.000019
80%|██████    | 4/5 [11:38<02:56, 176.94s/it]
Epoch 4/5 | Train Loss: 0.000018 | Val Loss: 0.000014
100%|███████  | 5/5 [14:48<00:00, 177.62s/it]
Epoch 5/5 | Train Loss: 0.000016 | Val Loss: 0.000013
Train MAE: 1.43 kcal/mol | Val MAE: 1.44 kcal/mol

```

In [12]: `import pandas as pd`

```

df_results = pd.DataFrame(results)
df_results = df_results.sort_values(by="val_mae").reset_index(drop=True)
df_results

```

Out [12]:

	hidden_dim	learning_rate	l2	final_val_loss	train_mae	val_mae
0	64	0.0010	0.00000	0.000005	0.979937	0.987342
1	256	0.0010	0.00000	0.000005	0.983503	0.994686
2	128	0.0010	0.00000	0.000006	1.146211	1.152172
3	256	0.0001	0.00000	0.000011	1.189181	1.201160
4	128	0.0001	0.00000	0.000010	1.326049	1.346883
5	64	0.0001	0.00001	0.000013	1.425070	1.433479
6	256	0.0001	0.00001	0.000013	1.431747	1.442144
7	64	0.0001	0.00000	0.000014	1.515486	1.535375
8	128	0.0010	0.00001	0.000015	1.570841	1.579133
9	128	0.0001	0.00001	0.000017	1.688814	1.693933
10	256	0.0010	0.00001	0.000019	1.720675	1.723294
11	64	0.0010	0.00001	0.000019	1.720143	1.728336

```
In [13]: best_config = df_results.loc[df_results["val_mae"].idxmin()]
print("Best configuration:")
print(best_config.to_string())
```

Best configuration:

```
hidden_dim      64.000000
learning_rate    0.001000
l2               0.000000
final_val_loss   0.000005
train_mae        0.979937
val_mae          0.987342
```

```
In [11]: # best_hidden = int(best_config["hidden_dim"])
# best_lr = float(best_config["learning_rate"])
# best_l2 = float(best_config["l2"])
# OUTPUTS WERE OBTAINED FROM LAST SAVIO SESSION

best_hidden = 64
best_lr = 1e-3
best_l2 = 0

# Rebuild model
net_H = AtomicNet(best_hidden)
net_C = AtomicNet(best_hidden)
net_N = AtomicNet(best_hidden)
net_O = AtomicNet(best_hidden)

ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
best_model = nn.Sequential(aev_computer, ani_net).to(device)

trainer = ANITrainer(
    best_model,
```

```

    batch_size=256,
    learning_rate=best_lr,
    epoch=5,
    l2=best_l2
)

train_loss, val_loss = trainer.train(
    train_data,
    val_data,
    draw_curve=True
)

```

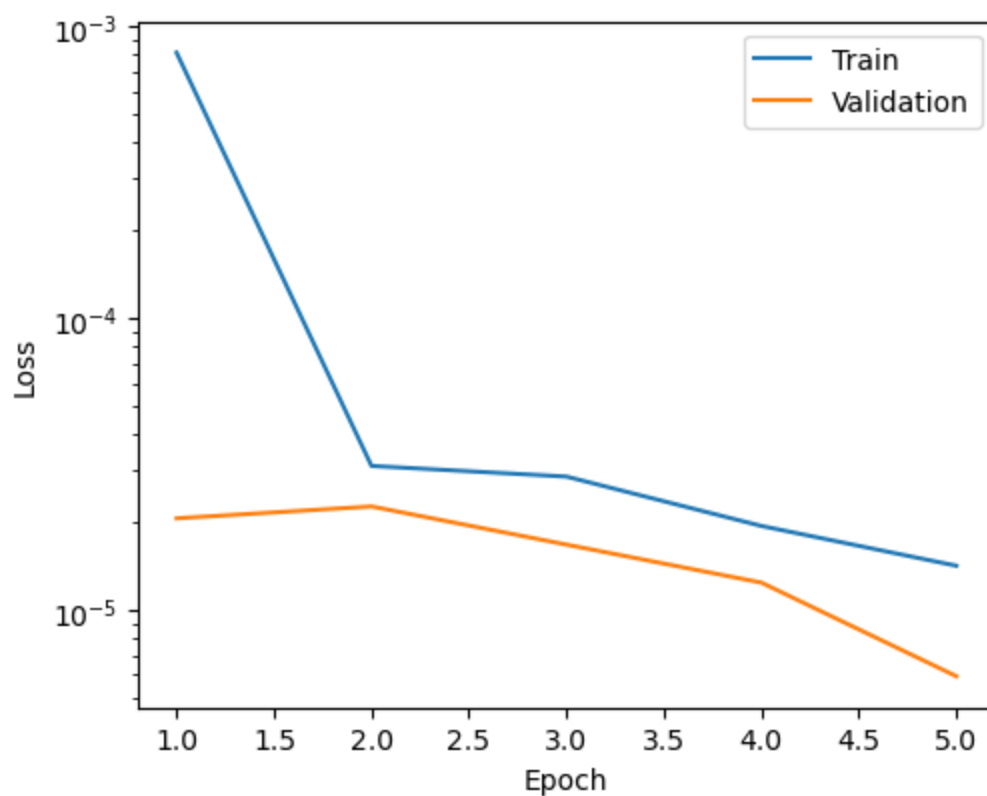
Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [01:46<07:05, 106.27s/it]
Epoch 1/5 | Train Loss: 0.000812 | Val Loss: 0.000021
40%|██████    | 2/5 [03:34<05:22, 107.46s/it]
Epoch 2/5 | Train Loss: 0.000031 | Val Loss: 0.000023
60%|██████    | 3/5 [05:21<03:34, 107.43s/it]
Epoch 3/5 | Train Loss: 0.000029 | Val Loss: 0.000017
80%|██████    | 4/5 [07:08<01:47, 107.27s/it]
Epoch 4/5 | Train Loss: 0.000019 | Val Loss: 0.000012
100%|██████   | 5/5 [08:58<00:00, 107.64s/it]
Epoch 5/5 | Train Loss: 0.000014 | Val Loss: 0.000006

```

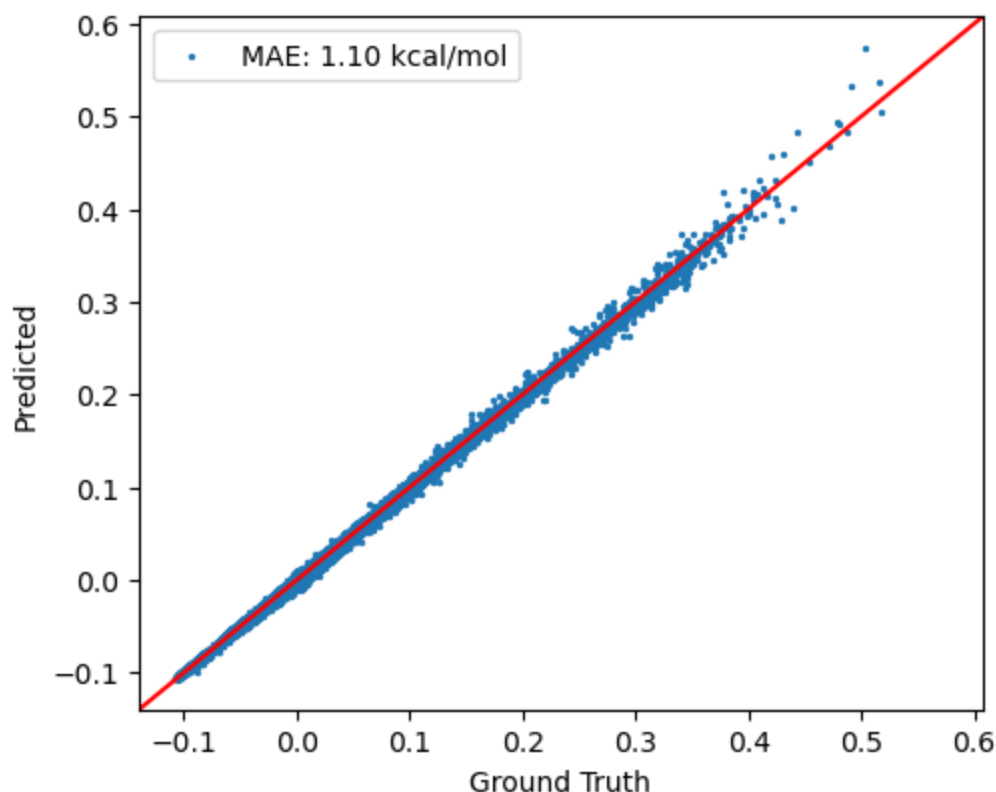


```

In [12]: test_mae = trainer.evaluate(test_data, draw_plot=True, return_mae=True)
         print(f"Test MAE: {test_mae:.2f} kcal/mol")

```

Test MAE: 1.10 kcal/mol



Checkpoint 3 evaluated multiple combinations of model architecture(64, 128, 256), learning rate($1e-3$ and $1e-4$), and L2 regularization(0 and $1e-5$) using more data than in previous checkpoints.

Based on the outputs, the best-performing model is the one with hidden dimension of 64, a learning rate of $1e-3$, and no L2 regularization because configuration produced the lowest validation MAE (approximately 0.99 kcal/mol). Increasing the model size to 128 or 256 did not improve performance and sometimes led to worse generalization. Also, adding L2 regularization slightly increased validation error, which suggest that regularization was not necessary good for this model and dataset. The results also showing that learning rate of $1e-3$ performed better than $1e-4$, which suggest that higher learning rate helped the model converge more effectively.

The model of my choice:

- hidden_dim = 64
- learning_rate = $1e-3$
- l2 = 0

Since this model achieved the lowest validation error, I retrained the model using these hyperparameters. Both training and validation losses decreased smoothly, indicating stable learning without overfitting, which demonstrated good generalization to unseen data. The model was then evaluated on the test set, and the MAE is around 1.10 kcal/mol. The predicted vs. true energy plot shows that most points lie close to $y = x$ (perfect prediction), indicating that the predicted values are very close to the true values, so the

model is making accurate predictions with low error. That's why i chose this model(hidden_dim = 64, learning_rate = 1e-3, l2 = 0) as the final model.

Checkpoint4: Final production mode (Multiple runs and N-fold cross-validation)

Best configuration from Checkpoint 3:

- hidden_dim = 64
- learning_rate = 1e-3
- l2 = 0

Multiple runs (5 runs conducted):

```
In [10]: def build_ani_model(hidden_dim):
net_H = AtomicNet(hidden_dim)
net_C = AtomicNet(hidden_dim)
net_N = AtomicNet(hidden_dim)
net_O = AtomicNet(hidden_dim)

ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(aev_computer, ani_net).to(device)
return model

In [11]: def run_single_training(train_data, val_data, test_data,
                                hidden_dim=64, learning_rate=1e-3, l2=0,
                                batch_size=256, epochs=5,
                                draw_curve=False):

    model = build_ani_model(hidden_dim)

    trainer = ANITrainer(
        model=model,
        batch_size=batch_size,
        learning_rate=learning_rate,
        epoch=epochs,
        l2=l2
    )

    train_losses, val_losses = trainer.train(
        train_data=train_data,
        val_data=val_data,
        draw_curve=draw_curve
    )

    train_mae = trainer.evaluate(train_data, return_mae=True)
    val_mae = trainer.evaluate(val_data, return_mae=True)
    test_mae = trainer.evaluate(test_data, return_mae=True)

    return {
        "model": model,
```

```

        "trainer": trainer,
        "train_losses": train_losses,
        "val_losses": val_losses,
        "train_mae": train_mae,
        "val_mae": val_mae,
        "test_mae": test_mae
    }

```

```

In [12]: num_runs = 5

multi_run_results = []

for run_idx in range(num_runs):
    print(f"\n===== Run {run_idx + 1}/{num_runs} =====")

    result = run_single_training(
        train_data=train_data,
        val_data=val_data,
        test_data=test_data,
        hidden_dim=64,
        learning_rate=1e-3,
        l2=0,
        batch_size=256,
        epochs=5,
        draw_curve=False
    )

    multi_run_results.append({
        "run": run_idx + 1,
        "train_mae": result["train_mae"],
        "val_mae": result["val_mae"],
        "test_mae": result["test_mae"]
    })

```

===== Run 1/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:29<01:59, 29.86s/it]
Epoch 1/5 | Train Loss: 0.000746 | Val Loss: 0.000023
40%|██████    | 2/5 [00:55<01:21, 27.22s/it]
Epoch 2/5 | Train Loss: 0.000024 | Val Loss: 0.000045
60%|██████    | 3/5 [01:23<00:55, 27.55s/it]
Epoch 3/5 | Train Loss: 0.000022 | Val Loss: 0.000025
Epoch 4/5 | Train Loss: 0.000016 | Val Loss: 0.000030
100%|██████████| 5/5 [02:15<00:00, 27.07s/it]
Epoch 5/5 | Train Loss: 0.000012 | Val Loss: 0.000030

```

===== Run 2/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:24<01:37, 24.34s/it]
Epoch 1/5 | Train Loss: 0.000214 | Val Loss: 0.000169
100%|██████████| 5/5 [02:09<00:00, 25.96s/it]
Epoch 5/5 | Train Loss: 0.000010 | Val Loss: 0.000018

```

===== Run 3/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:24<01:36, 24.06s/it]
Epoch 1/5 | Train Loss: 0.000349 | Val Loss: 0.000139
40%|██████    | 2/5 [00:47<01:11, 23.93s/it]
Epoch 2/5 | Train Loss: 0.000039 | Val Loss: 0.000056
60%|██████    | 3/5 [01:11<00:47, 23.81s/it]
Epoch 3/5 | Train Loss: 0.000025 | Val Loss: 0.000122
80%|██████    | 4/5 [01:41<00:26, 26.34s/it]
Epoch 4/5 | Train Loss: 0.000018 | Val Loss: 0.000025
100%|██████████| 5/5 [02:09<00:00, 25.89s/it]
Epoch 5/5 | Train Loss: 0.000014 | Val Loss: 0.000011

```

===== Run 4/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:24<01:36, 24.11s/it]
Epoch 1/5 | Train Loss: 0.000572 | Val Loss: 0.000027
40%|██████    | 2/5 [00:51<01:18, 26.13s/it]
Epoch 2/5 | Train Loss: 0.000036 | Val Loss: 0.000014
60%|██████    | 3/5 [01:14<00:49, 24.84s/it]
Epoch 3/5 | Train Loss: 0.000028 | Val Loss: 0.000049
80%|██████    | 4/5 [01:41<00:25, 25.50s/it]
Epoch 4/5 | Train Loss: 0.000020 | Val Loss: 0.000021
100%|██████████| 5/5 [02:06<00:00, 25.22s/it]
Epoch 5/5 | Train Loss: 0.000015 | Val Loss: 0.000007

```

===== Run 5/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:24<01:39, 24.79s/it]
Epoch 1/5 | Train Loss: 0.000164 | Val Loss: 0.000064
40%|██████    | 2/5 [00:53<01:20, 26.90s/it]
Epoch 2/5 | Train Loss: 0.000024 | Val Loss: 0.000011
60%|██████    | 3/5 [01:18<00:52, 26.21s/it]
Epoch 3/5 | Train Loss: 0.000015 | Val Loss: 0.000025
80%|██████    | 4/5 [01:45<00:26, 26.44s/it]
Epoch 4/5 | Train Loss: 0.000011 | Val Loss: 0.000016
100%|██████████| 5/5 [02:10<00:00, 26.10s/it]
Epoch 5/5 | Train Loss: 0.000009 | Val Loss: 0.000009

```

```

In [13]: import pandas as pd

df_multi_runs = pd.DataFrame(multi_run_results)
df_multi_runs

```

```
Out [13]:
```

	run	train_mae	val_mae	test_mae
0	1	2.054431	2.050165	2.054231
1	2	2.229321	2.226996	2.232416
2	3	1.662945	1.660891	1.671328
3	4	1.249773	1.244688	1.258913
4	5	1.578277	1.576451	1.581804

```
In [14]: print("Multiple-run summary:")
print(f"Mean train MAE: {df_multi_runs['train_mae'].mean():.3f} ± {df_multi_runs['train_mae'].std():.3f} kcal/mol")
print(f"Mean val MAE: {df_multi_runs['val_mae'].mean():.3f} ± {df_multi_runs['val_mae'].std():.3f} kcal/mol")
print(f"Mean test MAE: {df_multi_runs['test_mae'].mean():.3f} ± {df_multi_runs['test_mae'].std():.3f} kcal/mol")
```

Multiple-run summary:
Mean train MAE: 1.755 ± 0.390 kcal/mol
Mean val MAE: 1.752 ± 0.391 kcal/mol
Mean test MAE: 1.760 ± 0.387 kcal/mol

N-fold cross-validation (5-fold used):

```
In [15]: dev_data_list = list(train_data) + list(val_data)
print("Number of development samples:", len(dev_data_list))
print("Number of test samples:", len(list(test_data)))
```

Number of development samples: 778407
Number of test samples: 86491

```
In [16]: from sklearn.model_selection import KFold

k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)
```

```
In [17]: cv_results = []

for fold_idx, (train_idx, val_idx) in enumerate(kf.split(dev_data_list), start=1):
    print(f"\n===== Fold {fold_idx}/{k} =====")

    fold_train_list = [dev_data_list[i] for i in train_idx]
    fold_val_list = [dev_data_list[i] for i in val_idx]

    fold_train_data = torchani.data.TransformableIterable(fold_train_list)
    fold_val_data = torchani.data.TransformableIterable(fold_val_list)

    model = build_anl_model(hidden_dim=64)

    trainer = ANITrainer(
        model=model,
        batch_size=256,
```

```

        learning_rate=1e-3,
        epoch=5,
        l2=0
    )

    train_losses, val_losses = trainer.train(
        train_data=fold_train_data,
        val_data=fold_val_data,
        draw_curve=False
    )

    fold_train_mae = trainer.evaluate(fold_train_data, return_mae=True)
    fold_val_mae = trainer.evaluate(fold_val_data, return_mae=True)

    cv_results.append({
        "fold": fold_idx,
        "train_mae": fold_train_mae,
        "val_mae": fold_val_mae
    })

```

===== Fold 1/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:38<02:34, 38.61s/it]
Epoch 1/5 | Train Loss: 0.000254 | Val Loss: 0.000022
40%|██████    | 2/5 [01:01<01:27, 29.29s/it]
Epoch 2/5 | Train Loss: 0.000032 | Val Loss: 0.000022
60%|██████    | 3/5 [01:27<00:55, 27.75s/it]
Epoch 3/5 | Train Loss: 0.000022 | Val Loss: 0.000017
80%|██████    | 4/5 [02:02<00:30, 30.68s/it]
Epoch 4/5 | Train Loss: 0.000015 | Val Loss: 0.000010
100%|███████  | 5/5 [02:30<00:00, 30.01s/it]
Epoch 5/5 | Train Loss: 0.000011 | Val Loss: 0.000006

```

===== Fold 2/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:26<01:44, 26.24s/it]
Epoch 1/5 | Train Loss: 0.000618 | Val Loss: 0.000025
40%|██████    | 2/5 [00:52<01:19, 26.49s/it]
Epoch 2/5 | Train Loss: 0.000035 | Val Loss: 0.000019
60%|██████    | 3/5 [01:15<00:49, 24.86s/it]
Epoch 3/5 | Train Loss: 0.000031 | Val Loss: 0.000013
Epoch 4/5 | Train Loss: 0.000023 | Val Loss: 0.000010
100%|███████  | 5/5 [02:18<00:00, 27.61s/it]
Epoch 5/5 | Train Loss: 0.000017 | Val Loss: 0.000012

```

===== Fold 3/5 =====

Sequential – Number of parameters: 98820

Initialize training data...

```

20%|██████    | 1/5 [00:26<01:44, 26.05s/it]
Epoch 1/5 | Train Loss: 0.000727 | Val Loss: 0.000029
Epoch 2/5 | Train Loss: 0.000056 | Val Loss: 0.000027
60%|██████    | 3/5 [01:22<00:54, 27.50s/it]
Epoch 3/5 | Train Loss: 0.000040 | Val Loss: 0.000035

```

```

80%|██████████ | 4/5 [01:48<00:26, 26.68s/it]
Epoch 4/5 | Train Loss: 0.000031 | Val Loss: 0.000020
100%|██████████| 5/5 [02:10<00:00, 26.01s/it]
Epoch 5/5 | Train Loss: 0.000022 | Val Loss: 0.000011
===== Fold 4/5 =====
Sequential - Number of parameters: 98820
Initialize training data...
20%|██████ | 1/5 [00:27<01:51, 27.90s/it]
Epoch 1/5 | Train Loss: 0.000419 | Val Loss: 0.000021
40%|██████ | 2/5 [00:55<01:23, 27.98s/it]
Epoch 2/5 | Train Loss: 0.000028 | Val Loss: 0.000018
60%|██████ | 3/5 [01:18<00:50, 25.37s/it]
Epoch 3/5 | Train Loss: 0.000023 | Val Loss: 0.000009
Epoch 4/5 | Train Loss: 0.000016 | Val Loss: 0.000009
100%|██████████| 5/5 [02:23<00:00, 28.79s/it]
Epoch 5/5 | Train Loss: 0.000012 | Val Loss: 0.000007
===== Fold 5/5 =====
Sequential - Number of parameters: 98820
Initialize training data...
20%|██████ | 1/5 [00:23<01:33, 23.46s/it]
Epoch 1/5 | Train Loss: 0.000792 | Val Loss: 0.000028
40%|██████ | 2/5 [00:47<01:11, 23.68s/it]
Epoch 2/5 | Train Loss: 0.000026 | Val Loss: 0.000020
60%|██████ | 3/5 [01:22<00:58, 29.07s/it]
Epoch 3/5 | Train Loss: 0.000025 | Val Loss: 0.000016
80%|██████ | 4/5 [01:46<00:26, 26.85s/it]
Epoch 4/5 | Train Loss: 0.000018 | Val Loss: 0.000028
100%|██████████| 5/5 [02:11<00:00, 26.28s/it]
Epoch 5/5 | Train Loss: 0.000014 | Val Loss: 0.000014

```

```
In [19]: df_cv = pd.DataFrame(cv_results)
df_cv
```

```
Out[19]:
```

	fold	train_mae	val_mae
0	1	1.079807	1.085786
1	2	1.327407	1.330063
2	3	1.523374	1.532403
3	4	1.243466	1.248134
4	5	1.998620	1.996871

```
In [20]: print("Cross-validation summary:")
print(f"Mean train MAE: {df_cv['train_mae'].mean():.3f} ±
{df_cv['train_mae'].std():.3f} kcal/mol")
print(f"Mean val MAE: {df_cv['val_mae'].mean():.3f} ±
{df_cv['val_mae'].std():.3f} kcal/mol")
```

Cross-validation summary:
 Mean train MAE: 1.435 ± 0.354 kcal/mol
 Mean val MAE: 1.439 ± 0.351 kcal/mol

Summary:

```
In [21]: summary = pd.DataFrame({
    "Experiment": ["Multiple Runs", "5-Fold CV"],
    "mean_Train_MAE": [
        df_multi_runs["train_mae"].mean(),
        df_cv["train_mae"].mean()
    ],
    "std_Train_MAE": [
        df_multi_runs["train_mae"].std(),
        df_cv["train_mae"].std()
    ],
    "mean_Val_MAE": [
        df_multi_runs["val_mae"].mean(),
        df_cv["val_mae"].mean()
    ],
    "std_Val_MAE": [
        df_multi_runs["val_mae"].std(),
        df_cv["val_mae"].std()
    ],
    "mean_Test_MAE": [
        df_multi_runs["test_mae"].mean(),
        None
    ],
    "std_Test_MAE": [
        df_multi_runs["test_mae"].std(),
        None
    ]
})

summary
```

```
Out [21]:
```

	Experiment	mean_Train_MAE	std_Train_MAE	mean_Val_MAE	std_Val_MAE	mean
0	Multiple Runs	1.754949	0.390384	1.751838	0.390835	
1	5-Fold CV	1.434535	0.353518	1.438651	0.351070	

The results in Checkpoint 4, using the best model from Checkpoint 3, show that it performs well in both repeated training and cross-validation settings since the selected ANI model remains below the project target of 2 kcal/mol.

In multiple-run experiment, we kept the same train, validation, and test split, but retrains the model 5 times. The only main thing changing is the random initialization and the randomness during training. Since the MAE is not exactly the same in every run, this tells us the model's final performance depends somewhat on training randomness. Even so, the model still stays below 2 kcal/mol.

In cross-validation experiment, the algorithm changed the train/validation split itself and each fold trains on a different subset of the data and validates on a different held-out fold, so the exact composition of the training and validation data changes. Under cross-validation, the model still performs fairly well and stays below 2 kcal/mol.

The multiple-run experiment shows that the model's performance changes slightly in the repeated trainings, which suggests some sensitivity to random initialization and optimization. The cross-validation experiment shows that the model still performs reasonably well when the training and validation split changes, which suggests that the learned patterns generalize well across different data splits. Overall, the model generalizes well enough to support the best configuration chosen from Checkpoint 3 and is sufficient for the final checkpoint.

Checkpoint5: Final results (Train model with all data and compare your results to what's reported in the paper)

```
In [14]: best_hidden_dim = 64
best_lr = 1e-3
best_l2 = 0

net_H = AtomicNet(hidden_dim=best_hidden_dim)
net_C = AtomicNet(hidden_dim=best_hidden_dim)
net_N = AtomicNet(hidden_dim=best_hidden_dim)
net_O = AtomicNet(hidden_dim=best_hidden_dim)

ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])

final_model = torch.nn.Sequential(
    aev_computer,
    ani_net
).to(device)

final_trainer = ANITrainer(
    final_model,
    batch_size=256,
    learning_rate=best_lr,
    epoch=20,
    l2=best_l2
)
```

Sequential – Number of parameters: 98820

Train on all data:

```
In [15]: final_train_loss, final_val_loss = final_trainer.train(
    dataset,
    None,
    early_stop=False,
```

```

    draw_curve=True
)

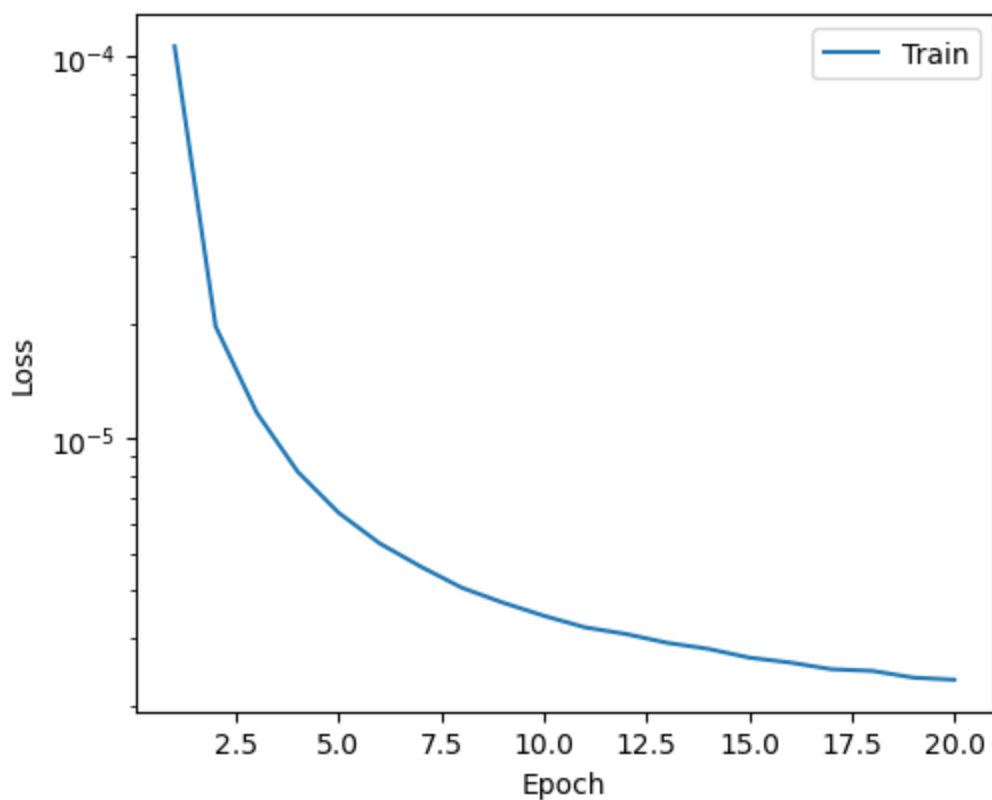
```

Initialize training data...

```

 5%|█          | 1/20 [00:23<07:25, 23.43s/it]
Epoch 1/20 | Train Loss: 0.000106
10%|██         | 2/20 [00:45<06:48, 22.70s/it]
Epoch 2/20 | Train Loss: 0.000020
15%|███        | 3/20 [01:06<06:14, 22.03s/it]
Epoch 3/20 | Train Loss: 0.000012
20%|████       | 4/20 [01:30<06:01, 22.61s/it]
Epoch 4/20 | Train Loss: 0.000008
25%|█████      | 5/20 [01:51<05:30, 22.03s/it]
Epoch 5/20 | Train Loss: 0.000006
30%|██████     | 6/20 [02:13<05:10, 22.21s/it]
Epoch 6/20 | Train Loss: 0.000005
35%|███████    | 7/20 [02:37<04:52, 22.54s/it]
Epoch 7/20 | Train Loss: 0.000005
40%|████████   | 8/20 [02:58<04:26, 22.24s/it]
Epoch 8/20 | Train Loss: 0.000004
45%|█████████  | 9/20 [03:21<04:06, 22.43s/it]
Epoch 9/20 | Train Loss: 0.000004
50%|██████████ | 10/20 [03:44<03:46, 22.63s/it]
Epoch 10/20 | Train Loss: 0.000003
55%|███████████| 11/20 [04:07<03:23, 22.56s/it]
Epoch 11/20 | Train Loss: 0.000003
60%|███████████| 12/20 [04:31<03:04, 23.10s/it]
Epoch 12/20 | Train Loss: 0.000003
65%|███████████| 13/20 [04:53<02:38, 22.67s/it]
Epoch 13/20 | Train Loss: 0.000003
70%|███████████| 14/20 [05:15<02:16, 22.70s/it]
Epoch 14/20 | Train Loss: 0.000003
75%|███████████| 15/20 [05:37<01:51, 22.33s/it]
Epoch 15/20 | Train Loss: 0.000003
80%|███████████| 16/20 [06:01<01:31, 22.86s/it]
Epoch 16/20 | Train Loss: 0.000003
85%|███████████| 17/20 [06:22<01:07, 22.47s/it]
Epoch 17/20 | Train Loss: 0.000002
90%|███████████| 18/20 [06:45<00:45, 22.54s/it]
Epoch 18/20 | Train Loss: 0.000002
95%|███████████| 19/20 [07:08<00:22, 22.69s/it]
Epoch 19/20 | Train Loss: 0.000002
100%|███████████| 20/20 [07:29<00:00, 22.47s/it]
Epoch 20/20 | Train Loss: 0.000002

```



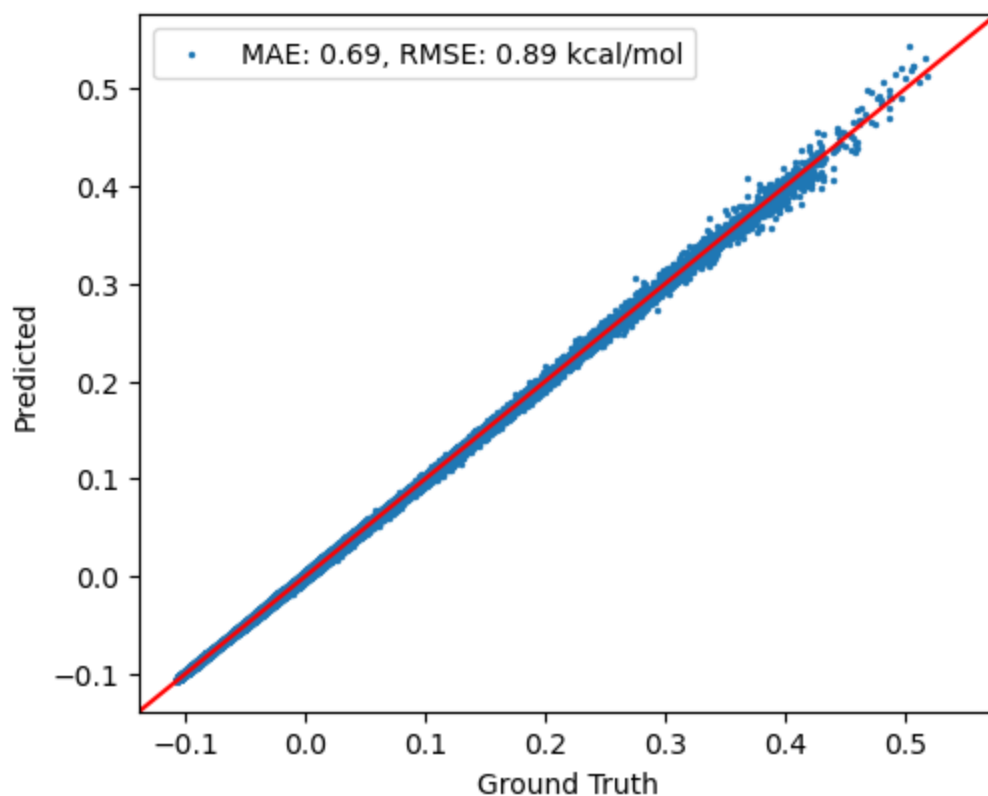
```
In [26]: print("ANI paper reported model-fit RMSE:")
print("Training RMSE:  1.2 kcal/mol")
print("Validation RMSE: 1.3 kcal/mol")
print("Test RMSE:      1.3 kcal/mol")
```

```
ANI paper reported model-fit RMSE:
Training RMSE:  1.2 kcal/mol
Validation RMSE: 1.3 kcal/mol
Test RMSE:      1.3 kcal/mol
```

```
In [27]: final_full_mae, final_full_rmse = final_trainer.evaluate(
    dataset,
    draw_plot=True,
    return_mae=True,
    return_rmse=True
)

print("Final full-data results:")
print(f"Final full-data MAE: {final_full_mae:.3f} kcal/mol")
print(f"Final full-data RMSE: {final_full_rmse:.3f} kcal/mol")
```

```
Final full-data results:
Final full-data MAE:  0.694 kcal/mol
Final full-data RMSE: 0.893 kcal/mol
```



The ANI paper reported model-fit RMSE values were 1.2 kcal/mol for training, 1.3 kcal/mol for validation, and 1.3 kcal/mol for testing. This result was obtained by training on 17.2 million conformations from about 58k molecules, using an 80/10/10 train/validation/test split. While the final training from checkpoint 5 obtained a full-data RMSE of 0.893 kcal/mol, lower than what was reported in the ANI paper, it is not an appropriate comparison since checkpoint 5 result was evaluated on the same full dataset used for final training and the paper reports split-based RMSE on a much larger and more diverse ANI-1 dataset. Therefore, a lower RMSE from checkpoint 5 doesn't mean the model is better than the model in the ANI paper, but it did learned the relationship between AEV molecular representations and reference energies effectively.